TIM2

72 MHz → 1 ms

每 1 ms 一次心跳

2026 纳新 · 教学部

机协教学部内训示范课件 · 储锐超 3260101888

机协教学部 · 内训示范课件

STM32 定时器与中断入门

从 “delay 阻塞” 到 “1 ms 心跳”

把延时交给硬件，把注意力留给真正的问题

主讲：储锐超 | 学号：3260101888

面向零基础：会一点点 C 就能跟上

CONTENTS

目录

01 **为什么需要定时器**
delay 把 CPU 占住了

02 **定时器怎么工作**
时钟 → 分频 → 计数 → 重装

03 **CubeMX 配置四步**
PSC / ARR / NVIC / 生成代码

04 **代码与中断回调**
只有两行启动代码

05 **五个常见坑**
volatile、回调名、别干重活

06 **课堂练习**
不用 delay 让 LED 慢闪

十分钟，六个问题
讲 5 分钟 + 动手 5 分钟

① 定时器是什么、能解决什么问题

② 时钟 / PSC / ARR 怎么配合

③ CubeMX 里四步配置

④ 回调函数与 tick 心跳

⑤ 五个最容易踩的坑

⑥ 课堂练习（改代码）

回顾：我们的第一个程序

点灯 + delay：HAL_Delay(500) 让 LED 每 500 ms 闪一次 —— 但这 500 ms 里，CPU 只是在“原地数圈”。

CPU 被占住

想同时读传感器、跑巡线，就再也塞不进别的任务。

时间不准

多个 delay 叠加后，实际周期越来越偏，越跑越飘。

不好扩展

每加一个任务，都要把整套时序重算一遍。

02 / 对比

阻塞式延时 vs 定时器中断

现在的做法：delay

CPU 原地空转
500 ms 里什么都干不了

只能串行做一件事
任务一多就互相打架

时间是“数循环”数出来的
主频一变就不准

我们要的做法：定时器中断

硬件在后台自动数数
不占用主循环时间

到点自动触发回调
到点就“敲一下”CPU

主循环专心做事
时间是硬件给的，很“硬”

一句话：把“等时间”这件事从 CPU 手里拿走，交给定时器。

01

定时器怎么工作

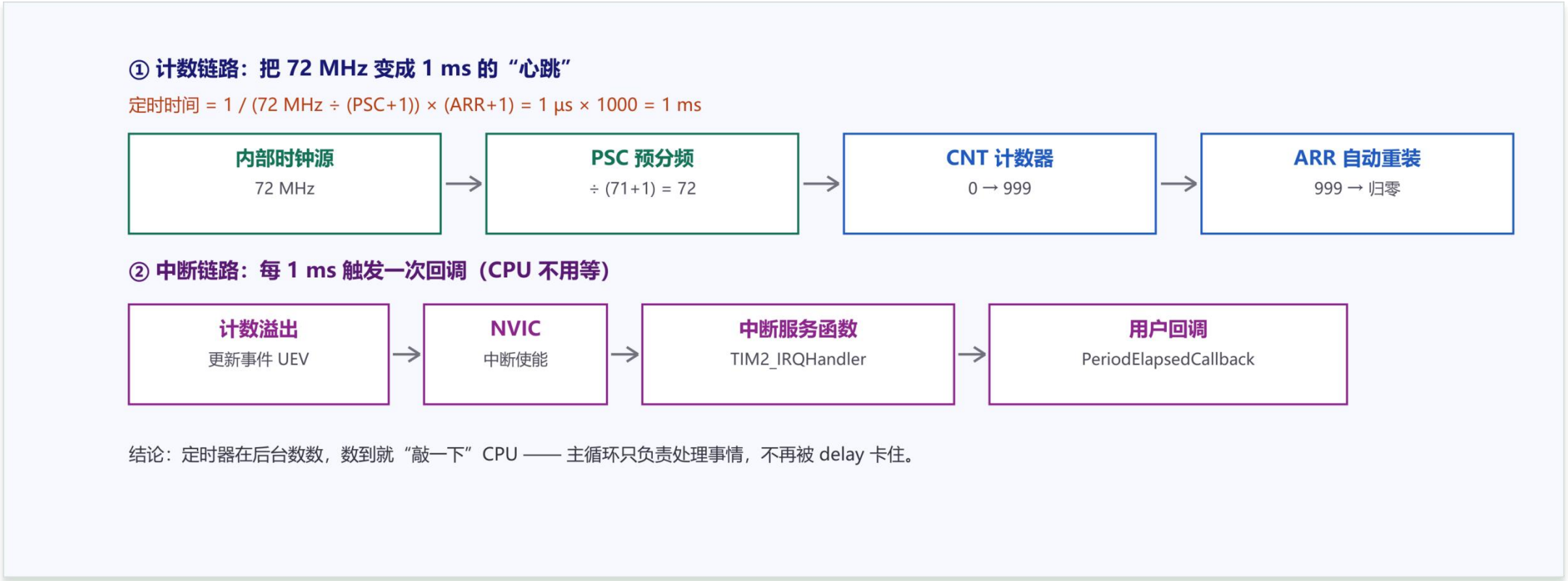
时钟 → 预分频 → 计数 → 重装 → 触发中断

一句话理解：硬件在后台不停地数数，数到约定的数就通知 CPU 一次。

我们要做的，只是把“数到多少”和“数到了做什么”告诉它。



定时器原理，一眼看懂



72 MHz 先被 PSC 分频成 1 MHz（1 µs 一格），CNT 数到 ARR 就溢出，触发更新中断，最终调用我们的回调函数。

三个数字，记住这一页就够

72 MHz

定时器时钟源

STM32F103 的 TIM2 挂在 APB1，倍频后 72 MHz

999

ARR 自动重装值

数到 999 溢出一次，即每 1000 个计数一次

1 ms

最终得到的周期

1 µs × 1000 = 1 ms，这就是我们的“心跳”

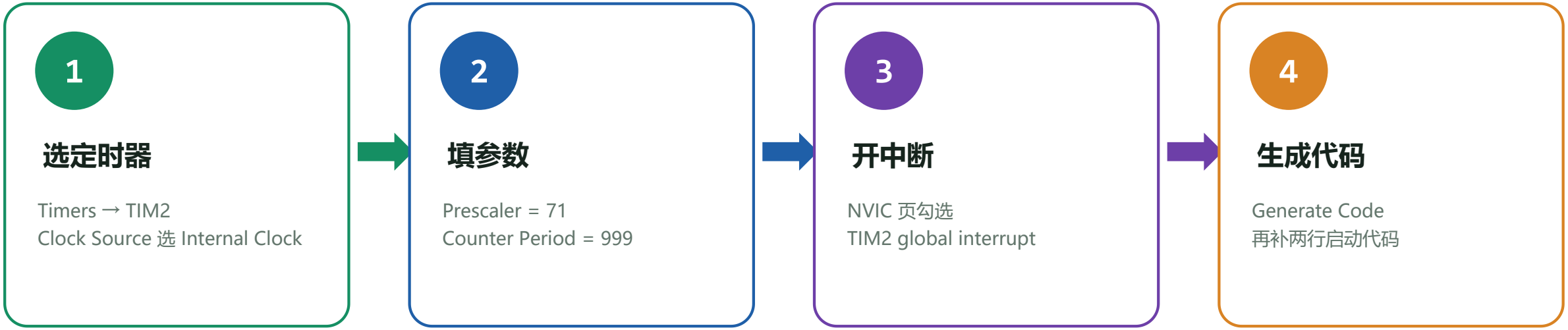
公式： 定时时间 = (PSC + 1) × (ARR + 1) ÷ 时钟频率

四个参数，一一对应

参数	作用	本例取值	怎么理解
时钟源	定时器的“心跳来源”	72 MHz	F103 的 TIM2 挂 APB1，倍频后 72 MHz
PSC 预分频	把时钟按比例变慢	71	分频系数 = PSC+1 = 72 → 1 MHz
ARR 自动重装	数到多少就溢出	999	溢出周期 = (ARR+1) × 1 μs = 1 ms
更新中断	溢出时通知 CPU	使能	NVIC 里勾选 TIM2 global interrupt

口诀：定时时间 = (PSC + 1) × (ARR + 1) ÷ 时钟频率；改 PSC 调“每格多长”，改 ARR 调“数多少格”。

CubeMX 里四步搞定



生成后你会得到：MX_TIM2_Init() 已经按你的参数配好，
你只需要在 USER CODE 区调用 HAL_TIM_Base_Start_IT(&htim2)，中断就活了。

代码其实只有两行

① 启动定时器

初始化完成后调用一次：
HAL_TIM_Base_Start_IT(&htim2);

② 中断回调

每 1 ms 自动被调用一次：
volatile uint32_t tick = 0;

回调里让 tick++ 即可，
其余交给硬件。

```c
/* tim.c —— CubeMX 生成工程后，在 USER CODE 区补充 */
#include "tim.h"

volatile uint32_t tick = 0;        // 全局心跳计数器（volatile 不能省）

HAL_TIM_Base_Start_IT(&htim2);     // 启动定时器并使能更新中断

/* 每 1 ms 自动调用一次的回调函数 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM2) {
        tick++;                    // 心跳 +1
    }
}
```

这就是“心跳”的全部代码——真正干活的只有 tick++ 一句。

主循环变得“非阻塞”

以前：delay 把时间占满

```
while (1) {
    HAL_GPIO_TogglePin(LED);
    HAL_Delay(500);   // ← CPU 在这里干等 500 ms
    // 想读传感器？只能排在这后面
```

问题：这段时间里 CPU 什么也做不了。

判据：从“等时间”变成“看时间”。

现在：用 tick 计时，主循环不阻塞

```
while (1) {
    if (tick - last >= 500) {
        last = tick;
        HAL_GPIO_TogglePin(LED);   // 到点才动手
    }
    // 还能顺便读传感器、跑巡线
```

同一块板子，同时做两件事。

五个常见坑（几乎人人踩过）

✕ **忘了 HAL_TIM_Base_Start_IT**
只初始化不启动，中断永远不会来。

✕ **回调函数名写错**
必须是 HAL_TIM_PeriodElapsedCallback，写错会静默失效。

✕ **tick 没加 volatile**
开了优化后编译器把它缓存进寄存器，主循环读到旧值。

✕ **在中断里干重活**
回调里别放串口打印、delay、长循环，否则主程序被拖慢。

✕ **PSC / ARR 算错**
记住公式再填，填完看一眼界面显示的周期对不对。

中断不是“打断你干活”，而是“让硬件替你数数”

—— CPU 只在该出手时出手。

机协教学部

课堂练习（5 分钟）

基础

把 LED 改成每 1 s 闪烁一次

要求：不许使用 HAL_Delay。想一想 tick 该和谁比较。

进阶

再加一个任务：每 2 s 打印一次 tick

两个任务用同一个 tick，观察它们是否互不干扰。

挑战

用 tick 量出“按键按了多久”

按下时记一次 tick，松开时再记一次，相减就是毫秒数。

写完可以对照下一节课的“答案代码”自查；遇到问题先看第 12 页的五个坑。

谢谢观看

把延时交给硬件，把注意力留给真正的问题

机协教学部 · 内训示范课件

主讲：储锐超　|　学号：3260101888

2026 纳新